Code559

Front-End Developer Course

MODULE 8

Arrays, Loops, and Objects

Keep many values in one place, visit each one in turn, and give each item named facts.

Free lesson: 559solutions.com/code559/module-08-arrays-loops-objects/

In this packet

1. **The lesson**
2. **Project: site-data.js**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to keep lists in arrays, visit every item with a loop, and give each item named facts with objects. It's the toolkit for any page built from data: twenty projects or milestones don't fit in twenty separate variables.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What is an array, and why does counting start at 0? 5
  - 2. What does `.length` tell you, and where is the last item? 5
  - 3. How do you add an item with `.push()`? 6
  - 4. How does a `for` loop visit every item? 7
  - 5. How do you use a condition inside a loop? 8
  - 6. When should you use `for...of` instead? 9
  - 7. How do `.includes()` and `.join()` help? 10
  - 8. What is the difference between a property and a method? 11
  - 9. What is an object? 12
  - 10. When does an array of objects beat parallel arrays? 14
  - 11. How do `&&`, `||` and `!` combine true and false? 14
- Exercise 15
- Common misconception 17
- Summary 17
- Check your understanding 18
- Before next module 18
- Go further 18
- What to keep 18

Part 2: Project: site-data.js 20

Part 3: Finishing work in Git 33

Part 4: Troubleshooting: five common problems 36

Part 5: Self-assessment 40

Part 6: Answers 42

About 559 Solutions 51

© 2026 559 Solutions LLC

PART 1

The lesson

Keep many values in one place, visit each one in turn, and give each item named facts.

Time: about 2.5 hours (estimate)

In this module you will learn:

- What an array is, and why its indexes start at 0
- How `.length` works, and why the last item is at `length - 1`
- How to add an item to the end with `.push()`
- The traditional `for` loop, and what `i++` does
- How to put a condition inside a loop to count or pick items
- The `for...of` loop, for when you only need each item
- How `.includes()` checks arrays and strings, and how `.join()` turns an array into text
- Properties versus methods: `object.property` versus `object.method()`
- Objects as named bundles of properties, dot access, and arrays of objects
- When an array of objects beats several parallel arrays
- The logical operators `&&`, `||` and `!`

Why this matters

A real site has several pages, several projects and several milestones. So far, every value in your JavaScript has lived in its own variable. That works for two or three values. It does not work for twenty, and it does not work when you do not know how many there will be.

This module gives you the three tools every data-driven page is built from. An **array** keeps a list in one variable. A **loop** visits every item in that list without you writing one line per item. An **object** gives each item named facts, like a title, a year and whether it is live. Put the three together and you have an array of objects, which is how most real web data is shaped.

You stay in the Console and Node one more time. In Module 9 you use the same patterns on a web page, where a loop over an array of objects builds a list a visitor can see.

Before you start

1. Finish Module 7. You should be able to write a function with a parameter and a `return`, and test it with several calls.
2. In your course folder, create a throwaway folder called `practice-08`. Open it in VS Code with **File → Open Folder...**, then open a terminal with **Terminal → New Terminal**.
3. Open Edge or Chrome. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab.

© 2026 559 Solutions LLC

4. On paper, write down three of your own interests and two projects or milestones you would like on your site. You will use them instead of the examples.

> **Checkpoint.** You have an empty `practice-08` folder open in VS Code, a terminal whose prompt ends in `practice-08>`, a Console ready, and your own short list on paper.

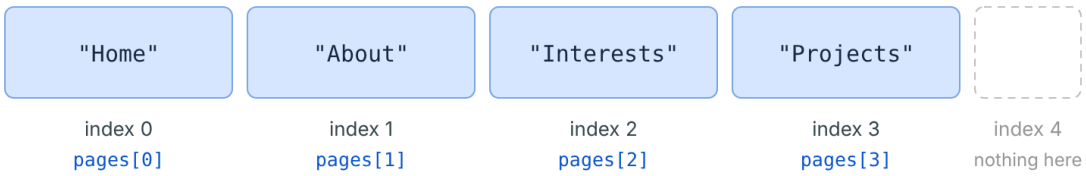
Core concepts

1. What is an array, and why does counting start at 0?

An array is one variable that holds a list of values, in order. You write it with square brackets and commas.

Each item has a position number called an **index**. The first index is `0`, not `1`. Think of the index as "how many steps from the start": the first item is zero steps away. You read an item with the array name and the index in square brackets: `pages[0]`.

```
const pages = ["Home", "About", "Interests", "Projects"];
```



Counting starts at 0. pages.length is 4, so the last index is pages.length - 1, which is 3. pages[4] reads past the end and gives undefined.

Four items, indexes 0 to 3. There is no index 4.

Before you type anything, think this through: what does `pages[2]` hold?

2. What does `.length` tell you, and where is the last item?

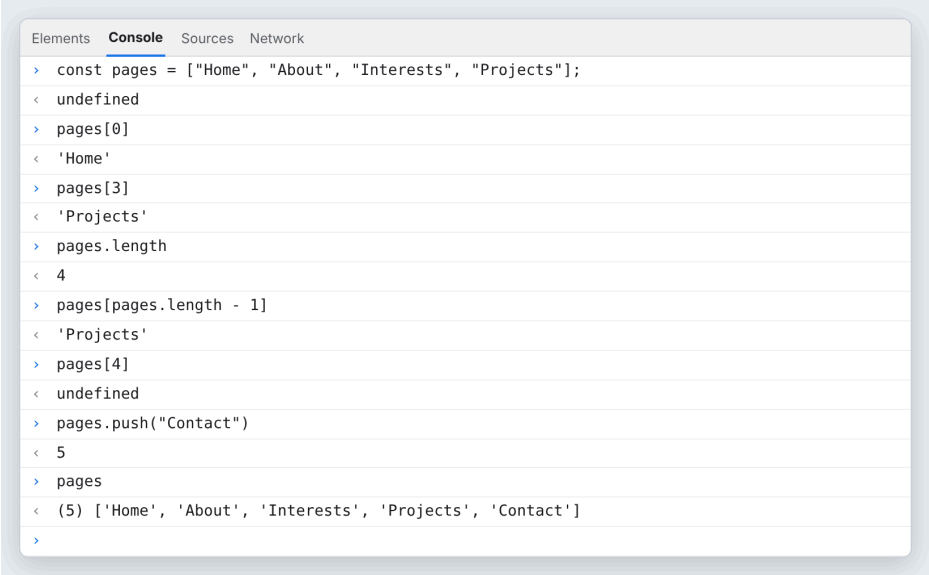
`.length` is the number of items in the array.

© 2026 559 Solutions LLC

Because counting starts at 0, the last index is always one less than the length. That is why `pages[pages.length - 1]` always gives the last item, however long the list grows. Reading at an index that does not exist is not an error. You get `undefined`, quietly.

Typed into the Console for the picture below, in this order:

```
const pages = ["Home", "About", "Interests", "Projects"];
pages[0]
pages[3]
pages.length
pages[pages.length - 1]
pages[4]
pages.push("Contact")
pages
```



`pages[4]` gives `undefined`, not an error. Note what `.push()` prints: `5`, the new length, not the array. (real output, rendered)

Type the `pages` array into your Console and read `pages[pages.length]`. Predict first. Why is it `undefined`?

3. How do you add an item with `.push()`?

`.push()` adds one or more items to the end of an array.

© 2026 559 Solutions LLC

The array changes in place. The value `.push()` hands back is the new length (MDN, *Array.prototype.push()*: "The new length property of the object upon which the method was called"). So do not write `pages = pages.push("Contact")`. That would try to replace your array with a number. Just call `pages.push("Contact")` on its own line. This works even though `pages` was made with `const`, because `const` stops you pointing the name at a different array; it does not stop the array's contents from changing.

In the image above, `pages.push("Contact")` printed `5`, and the next line showed five items with `'Contact'` at the end.

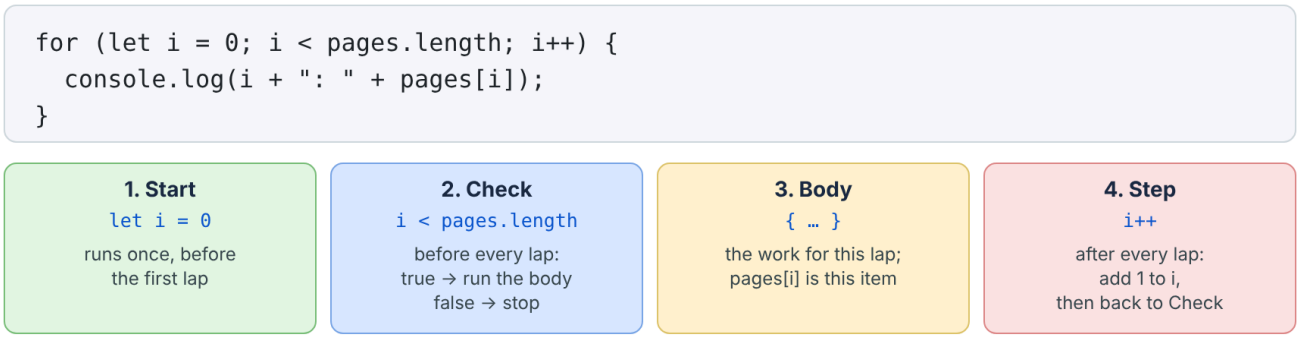
Push one of your own interests onto an array, then check `.length` went up by one.

4. How does a `for` loop visit every item?

A loop runs the same block of code again and again. Each run is called a **lap**, like one lap around a running track. The traditional `for` loop counts, so it is a natural fit for indexes.

The line `for (let i = 0; i < pages.length; i++)` has three parts between the parentheses, separated by semicolons. **Start**: `let i = 0` runs once. **Check**: `i < pages.length` runs before every lap; while it is `true`, the body runs; when it is `false`, the loop stops. **Step**: `i++` runs after every lap. `++` means "add 1 to this variable." It is short for `i = i + 1`. Inside the body, `pages[i]` is the item for this lap.

```
for (let i = 0; i < pages.length; i++) {
  console.log(i + ": " + pages[i]);
}
```



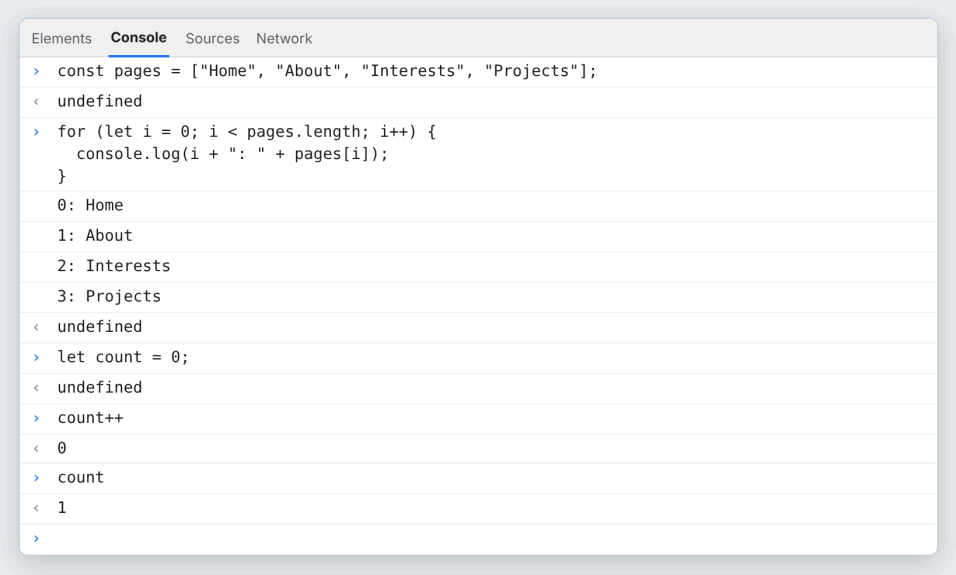
The check uses `<`, not `<=`. With 4 items, `i` takes the values 0, 1, 2 and 3, and the loop stops when `i` reaches 4.

Typed into the Console for the picture below, in this order. The first line makes a fresh four-item `pages`, without the `"Contact"` you pushed above. (The Console lets you type a `const` line again to start over; in a file, declaring the same name twice is an error.)

```
const pages = ["Home", "About", "Interests", "Projects"];
for (let i = 0; i < pages.length; i++) {
```


© 2026 559 Solutions LLC

```
  console.log(i + ": " + pages[i]);
}
let count = 0;
count++
count
```



Four laps, four lines. At the bottom, `count++` shows `0` because it hands back the old value and then adds 1. Asking for `count` on the next line shows the 1. Inside a loop you never use that handed-back value, so it does not matter there. (real output, rendered)

Change `i < pages.length` to `i < 2` and predict how many lines print.

5. How do you use a condition inside a loop?

Put an `if` inside the loop body, and the loop can count or pick only some items.

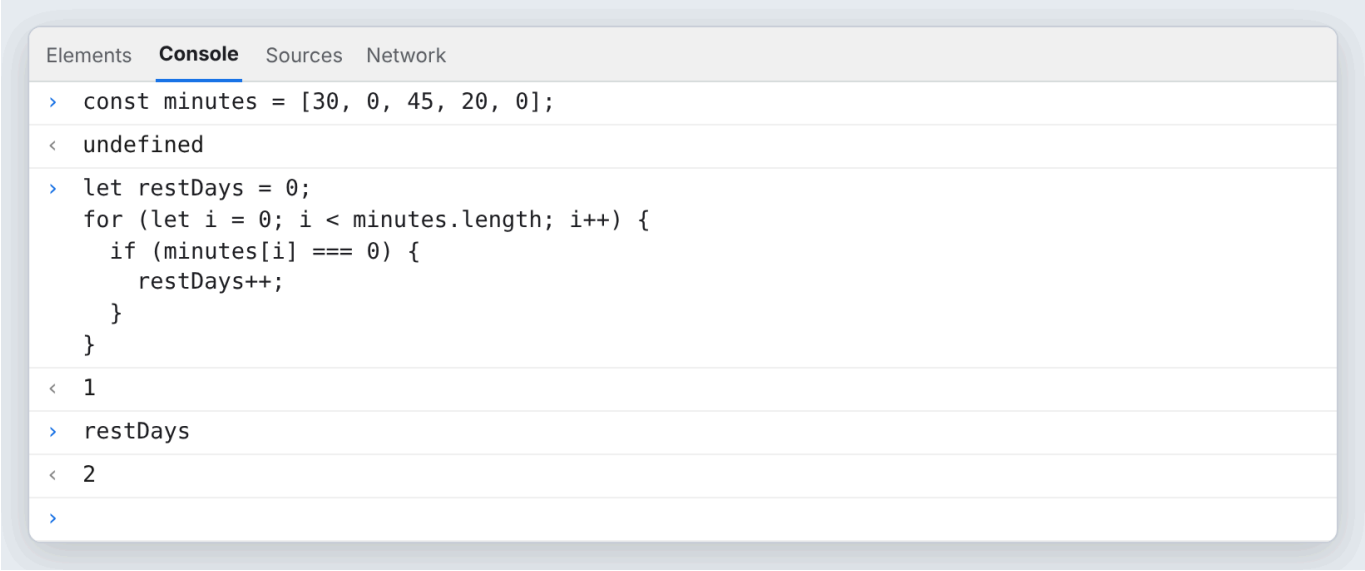
The pattern is: make a counter before the loop, check each item inside the loop, and add to the counter when the check is true. After the loop, the counter holds the answer. You used `if` in Module 6; nothing about it changes inside a loop.

Typed into the Console for the picture below, in this order. (In the picture, `let restDays = 0;` and the loop were typed as one entry. If you type them one at a time, you see one extra `undefined` after the first line.)

```
const minutes = [30, 0, 45, 20, 0];
let restDays = 0;
for (let i = 0; i < minutes.length; i++) {
```


© 2026 559 Solutions LLC

```
  if (minutes[i] === 0) {
    restDays++;
  }
}
restDays
```



An array of practice minutes is checked day by day, and every `0` adds one to `restDays`. The answer is `2`. The `1` right after the loop is not the answer: after any loop, the Console also shows a value left over from the last lap, often `undefined`. To see the answer, ask for the variable, as on the next line. (real output, rendered)

Change the check to `minutes[i] >= 30` and the name to `longDays`. Predict the answer before you run it. (It is 2: the 30 and the 45.)

6. When should you use `for...of` instead?

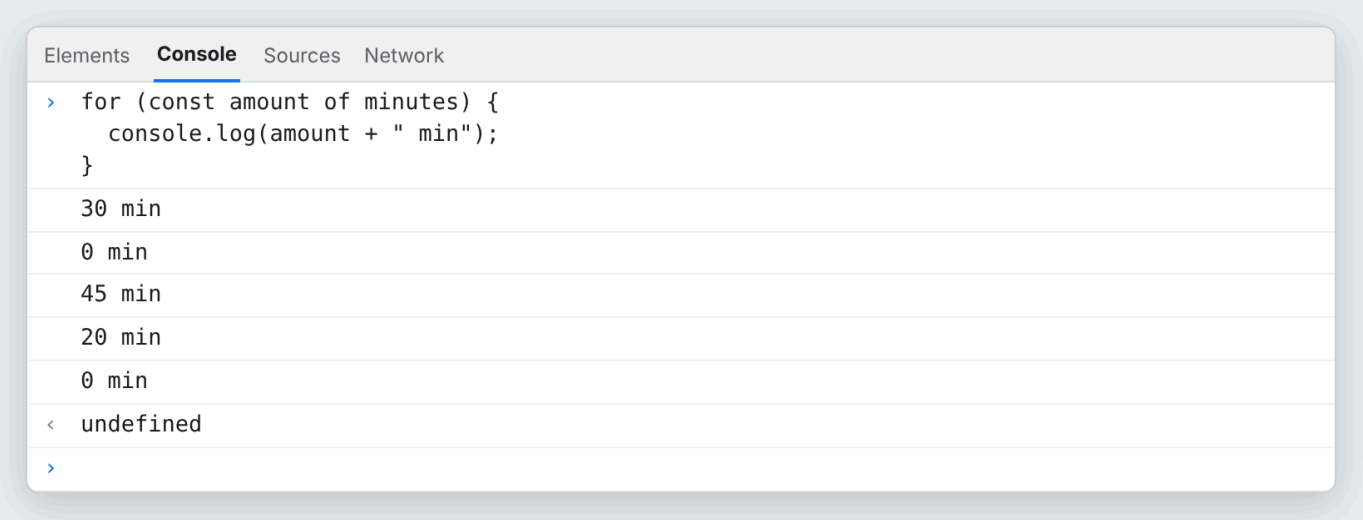
`for...of` is a second kind of loop. On an array, it runs one lap per item, in order, and puts the item itself in a variable for you.

© 2026 559 Solutions LLC

`for (const amount of minutes)` gives you each value directly, with no `i` and no square brackets. Use it when you only need the items. Use the counting `for` loop when you also need the position, for example to print `1:`, `2:`, `3:` beside each one.

Typed into the Console for the picture below, in this order, right after the Concept 5 lines (so `minutes` already exists):

```
for (const amount of minutes) {
  console.log(amount + " min");
}
```



`for...of` visits the same five values as the counting loop in Concept 5, with less to type. The `undefined` at the end is the value left over from the last lap, as in Concept 5. (real output, rendered)

Rewrite the rest-day counter with `for...of`. You need `if (amount === 0)` inside.

7. How do `.includes()` and `.join()` help?

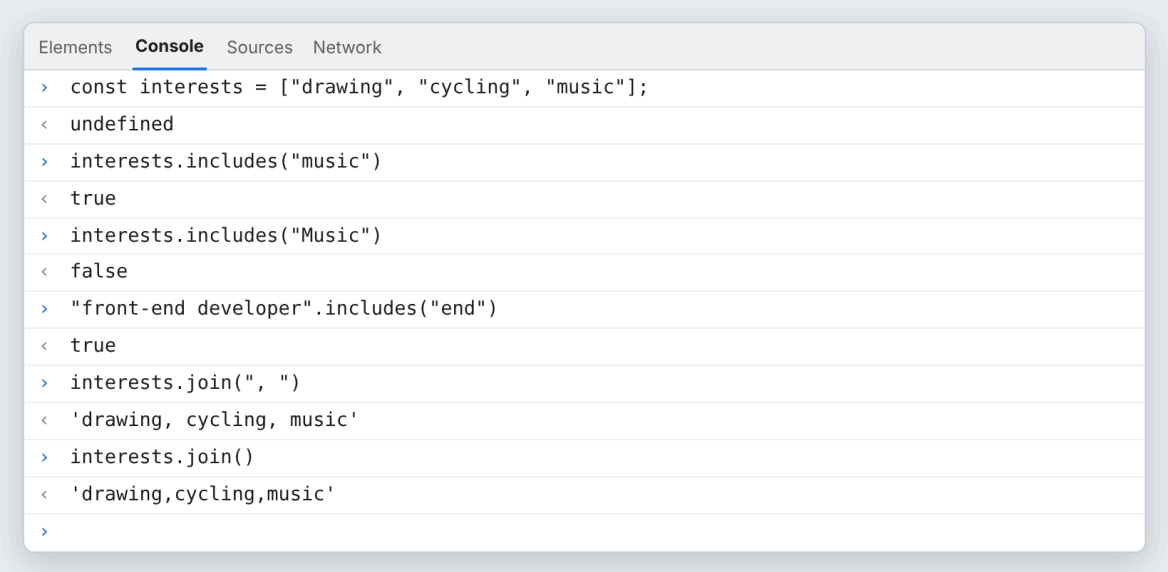
`.includes()` answers "is this in here?" with `true` or `false`. `.join()` turns an array into one piece of text.

© 2026 559 Solutions LLC

`.includes()` works on arrays and on strings. On an array, it checks whole items. On a string, it checks for a piece of text anywhere inside. It is exact: `"Music"` and `"music"` are different. `.join()` puts the text you give it between items. With nothing in the parentheses, MDN says the items "are separated with a comma."

Typed into the Console for the picture below, in this order:

```
const interests = ["drawing", "cycling", "music"];
interests.includes("music")
interests.includes("Music")
"front-end developer".includes("end")
interests.join(", ")
interests.join()
```



Capital letters matter to `.includes()`. And notice the difference between `join(", ")` and `join()`: the space after the comma only appears when you ask for it. (real output, rendered)

Join your own interests with `" / "` and with `" and "`. Which reads better as a sentence?

8. What is the difference between a property and a method?

Arrays and strings carry named values you can read, and named actions you can run. A value is a **property**. An action is a **method**.

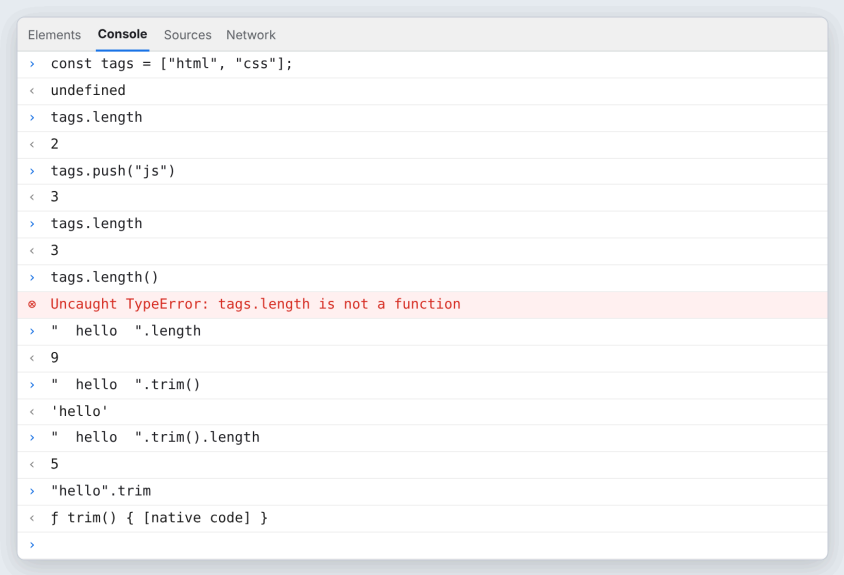
You read a property with a dot and its name: `tags.length`. You run a method with a dot, its name and parentheses: `tags.push("js")`, `" hello ".trim()`. MDN's glossary defines a method as "a function which is a property of an object." So this is Module 7 again: a method name without

© 2026 559 Solutions LLC

parentheses is a reference to the function; with parentheses it is a call. `.trim()` returns a copy of the string without the spaces at both ends; you will lean on it in Module 10 to check form inputs.

Typed into the Console for the picture below, in this order (two spaces before and after `hello`):

```
const tags = ["html", "css"];
tags.length
tags.push("js")
tags.length
tags.length()
"  hello  ".length
"  hello  ".trim()
"  hello  ".trim().length
"hello".trim
```



Parentheses on a property are an error, because a number cannot be run. A method without parentheses gives you the function itself, the same thing you saw with `showMessage` in Module 7. (real output, rendered)

For each of these, say "property" or "method" to yourself, out loud, before you type it:[1] `pages.length`, `pages.push("Blog")`, `"Hi ".trim()`, `"Hi ".length`.

9. What is an object?

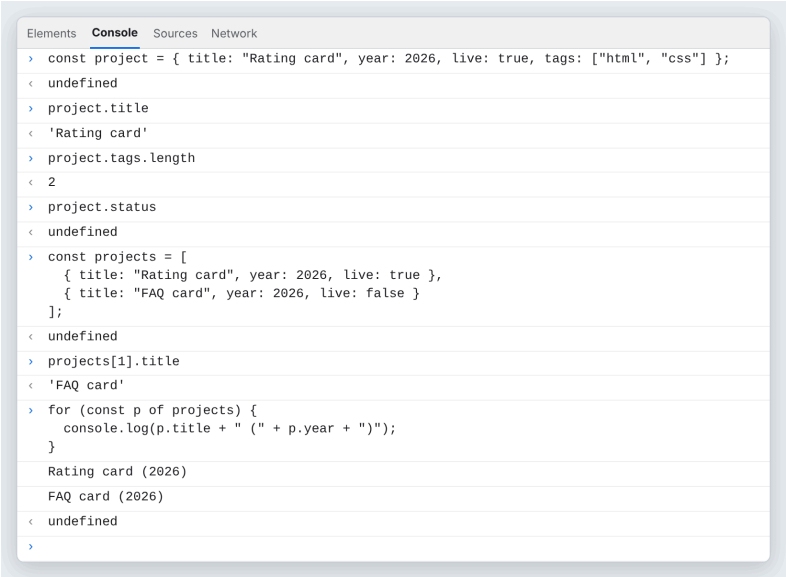
An object is a named bundle of properties. Where an array is a list in order, an object is a set of labeled facts about one thing.

© 2026 559 Solutions LLC

You write an object with curly braces. Each property is a name, a colon and a value, separated by commas: `{ title: "Rating card", year: 2026, live: true, tags: ["html", "css"] }`. You read a property with a dot: `project.title`. A value can be anything, including an array, so `project.tags.length` reads the length of the array stored in `tags`. Asking for a property that does not exist gives `undefined`, just like reading past the end of an array.

Typed into the Console for the picture below, in this order:

```
const project = { title: "Rating card", year: 2026, live: true, tags: ["html", "css"]
};
project.title
project.tags.length
project.status
const projects = [
  { title: "Rating card", year: 2026, live: true },
  { title: "FAQ card", year: 2026, live: false }
];
projects[1].title
for (const p of projects) {
  console.log(p.title + " (" + p.year + ")");
}
```



`projects[1].title` reads from left to right: the array, then item 1, then its `title`. The loop at the bottom visits each object and reads two of its properties. (real output, rendered)

Make an object for one of your own projects or milestones, with at least a `title` and a `year`. Read each property back.

© 2026 559 Solutions LLC

10. When does an array of objects beat parallel arrays?

Parallel arrays are several arrays where item 0 of each belongs together, item 1 of each belongs together, and so on.

They work until something changes. Delete an item from one array and forget the other, and every item after it is paired with the wrong facts, with no error to warn you. An array of objects keeps each item's facts together, so they cannot drift apart. Use an array of objects whenever each item has two or more facts.

Parallel arrays: related only by position

```
const titles = ["Rating card", "FAQ card"];
const years  = [2026, 2026];
const live   = [true, false];
```

Delete a title but forget to delete its year, and every title after it now pairs with the wrong year.

Array of objects: each item carries its own facts

```
const projects = [
  { title: "Rating card", year: 2026, live: true },
  { title: "FAQ card", year: 2026, live: false }
];
```

The same two projects, stored two ways. In the bottom version, adding a fact means adding one property, not a whole new array.

Take your paper list of projects or milestones and write it as an array of objects.

11. How do `&&`, `||` and `!` combine true and false?

A change of topic for the last concept: it is not about arrays, but the Exercise needs it to check two facts about each place at once.

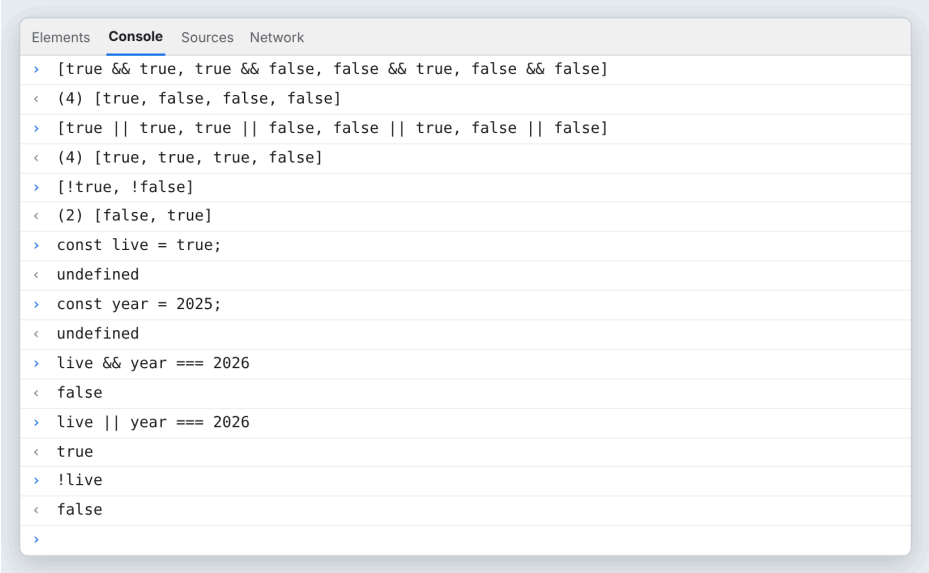
Logical operators join two checks into one, or flip one.

© 2026 559 Solutions LLC

`&&` (AND) is `true` only when **both** sides are `true`. `||` (OR) is `true` when **at least one** side is `true`. `!` (NOT) flips `true` to `false` and `false` to `true`. You will use them in `if` statements, such as "the project is live AND it is from this year." MDN adds one detail: with values that are not `true` or `false`, these operators hand back one of the values themselves (for example, `"Home" && "About"` gives `'About'`). In this course you will use them with `true` and `false`.

Typed into the Console for the picture below, in this order:

```
[true && true, true && false, false && true, false && false]
[true || true, true || false, false || true, false || false]
[!true, !false]
const live = true;
const year = 2025;
live && year === 2026
live || year === 2026
!live
```



The first three lines are the full truth tables. Only one `&&` row is `true`; only one `||` row is `false`. Then a real use: `year === 2026` is `false`, so the `&&` fails and the `||` passes. (real output, rendered)

Predict `!live && year === 2025`, then type it to check.

Exercise

Your hands, about 50 minutes (estimate). Type it, do not paste it. Work in `practice-08`. Use your own interests and places instead of the examples if you like; the shape is what matters.

© 2026 559 Solutions LLC

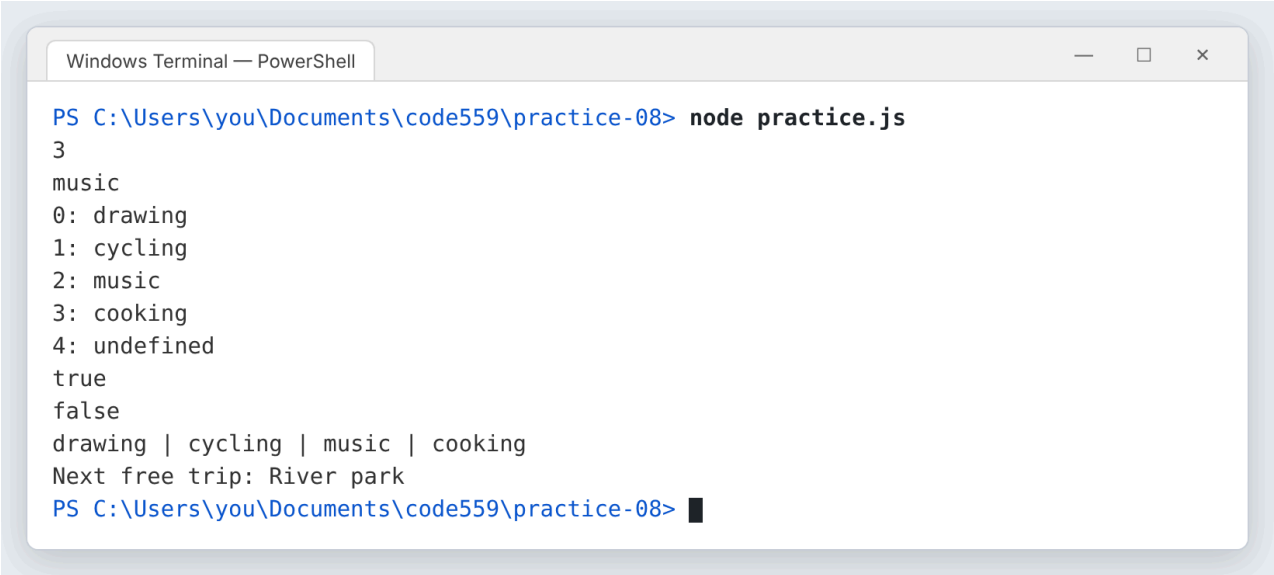
1. **Create `practice.js`** and type an array of three interests named `interests`, for example `const interests = ["drawing", "cycling", "music"];`. Below it, log its `.length` and its last item using `length - 1`.
2. **Push one more** interest with `.push()`.
3. **Loop with indexes.** Write a `for` loop that logs each item with its index in front, like `0: drawing`.
4. **Check and join.** Log `interests.includes(…)` for one item you have and one you do not. Then log `interests.join(" | ")`.
5. **An array of objects.** Type this, with your own place names if you prefer:

   ```
   const places = [
     { name: "City library", visited: true, free: true },
     { name: "Art museum", visited: false, free: false },
     { name: "River park", visited: false, free: true }
   ];

   for (const place of places) {
     if (!place.visited && place.free) {
       console.log("Next free trip: " + place.name);
     }
   }
   ```

   Read the `if` to yourself, out loud, before you run it: "not visited, and free."
6. **Run it.** Save, then type `node practice.js` in the terminal. Predict every line first. With the example data you should get 10 lines, ending in `Next free trip: River park`.
7. **Break it on purpose.** In the `for` loop from step 3, change `i < interests.length` to `i <= interests.length`. Save and run.

© 2026 559 Solutions LLC



One character added, one extra lap. On the last lap `i` is 4, which equals the length, and `interests[4]` is `undefined`. No error message, just a wrong line. This is called an off-by-one error. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Change `<=` back to `<`, run again, and check the `undefined` line is gone.

8. **Your own.** Below everything, write a loop that counts how many places are `free` and logs the count. With the example data, the answer is 2.

9. **Modify.** Change the `if` in step 5 from `&&` to `||`. Predict which places print now, then run it.

Common misconception

> *"An array with 4 items ends at index 4."*

What would you say? Hint: look at what `pages[4]` printed in the Concept 2 image, and count how many steps the first item is from the start.

Summary

An array keeps a list of values in order, and its first index is 0. `.length` is the number of items, so the last item is at `length - 1`, and reading past the end gives `undefined`. `.push()` adds to the end and hands back the new length. The traditional `for` loop has a start, a check and a step, and `i++` adds 1 to `i` after every lap. An `if` inside a loop lets you count or pick items. `for...of` gives you each item directly when you do not need its index. `.includes()` answers `true` or `false` for arrays and strings, exactly, and `.join()` turns an array into text with the separator you choose. A property is a value you

© 2026 559 Solutions LLC

read with `object.property`; a method is an action you run with `object.method()`. An object is a named bundle of properties read with dot access, and an array of objects keeps each item's facts together where parallel arrays can drift apart. `&&` needs both sides true, `||` needs one, and `!` flips a value.

Check your understanding

1. `tags.length()` throws a `TypeError`, but `tags.push("js")` works. Why does one need parentheses and the other must not have them?
2. You have six projects in an array. Which index holds the last one, and how would you write that so it still works when you add a seventh?
3. **What value does this variable hold** after this code runs? `let n = 0; for (const p of places) { if (p.visited || p.free) { n++; } }` (use the three places from the Exercise)

Before next module

- [ ] `practice.js` runs with `node practice.js` and every line matches your predictions
- [ ] You saw the extra `undefined` line from `<=` and fixed it
- [ ] You can say, for `.length`, `.push()` and `.trim()`, which is a property and which are methods
- [ ] Your own projects or milestones are written on paper (or in a file) as an array of objects
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *Array.prototype.push()*, *Array.prototype.includes()*, *Array.prototype.join()*
- MDN: *for...of*; and search "for statement" for the counting loop
- MDN: *Logical AND (&&)*, and the matching *Logical OR (||)* and *Logical NOT (!)* pages
- MDN Glossary: *Method*; and *String.prototype.trim()*
- freeCodeCamp: the free JavaScript Certification (checked 2026-10-11) has practice on this topic.

What to keep

Keep `practice.js` and your array of objects for your own projects or milestones; the Module 8 project starts from it. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

© 2026 559 Solutions LLC

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "Array.prototype.push()"; MDN "Array.prototype.includes()"; MDN "Array.prototype.join()"; MDN "for...of"; MDN "Logical AND (&&)"; MDN Glossary "Method"; MDN "String.prototype.trim()"; Chromium (via Playwright) for every Console result; Node 22.22.0 for every terminal result. Checked 2026-10-11: MDN "for...of" (plain-words summary); Chromium 141 for the split Concept 5 and 6 pictures (one Console session), the value shown after each loop, `project.tags.length` *and* `join(" / ")`*. Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).*

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: site-data.js

Your hands, about 90 minutes (estimate). In the Exercise you kept a list of interests in an array, visited each item with a loop, and picked places from an array of objects with `&&` and `!`. Now you do the same with data about you. The file is `site-data.js`. It holds your real projects and milestones as two arrays of objects, and five small functions that summarize them. Each function starts as a one-sentence description, and you write its tests **before** its body, as in Module 7. You use only what Modules 5 to 8 taught:

- comments, `console.log()`, `const` and `let`
- joining text with `+`, `-` to subtract, and the comparisons `===`, `>`, `<` and `<=`
- `if`, and functions with parameters, `return` and `null`
- arrays with an index from 0, `.length`, `.push()`, `.includes()` and `.join()`
- the counting `for` loop with `++`, and `for...of`
- objects with dot access, and `&&` and `!`

This project goes in `my-site`, next to `about-me.js`, `plan-my-day.js` and `helpers.js`. Any `my-site` that is a Git repository connected to GitHub works, because no page uses the file yet.

The brief

Read the whole brief twice before you type anything.

When you finish, `my-site` holds a new file, `site-data.js`, at the top level, next to `index.html`. No page links to it and no page runs it yet: like your other scripts, it runs in the terminal only. It does not use `helpers.js`. Each script in this course stands alone.

The data. Two arrays of objects, with your own real items in them.

- **`projects`**: the things you have built. Every packet project you finished counts: your Home and About pages, your Interests page, the rating card, your scripts. Something you made before this course counts too. Each project is an object with five properties, always in this order: `title` (text), `year` (a number, with no quotes), `description` (one short sentence), `tags` (an array of lowercase words such as `"html"`, `"css"` and `"js"`) and `live` (`true` when a visitor can open it as a page on your live site, `false` when not; a script runs in the terminal only, so it is `false`).
- **`milestones`**: things that happened on your way, such as the day your first page went live. Each milestone is an object with three properties: `title`, `year` and `description`. A milestone is not something you built, so it has no tags and nothing to put live.

Use at least four projects and three milestones. To give your tests something to find, include at least one project that is live and one that is not, two projects with the same tag, and two milestones from the same year.

© 2026 559 Solutions LLC

The five descriptions. Each function is one sentence. Use the names given here, so your tests and the answers in Part 6 line up.

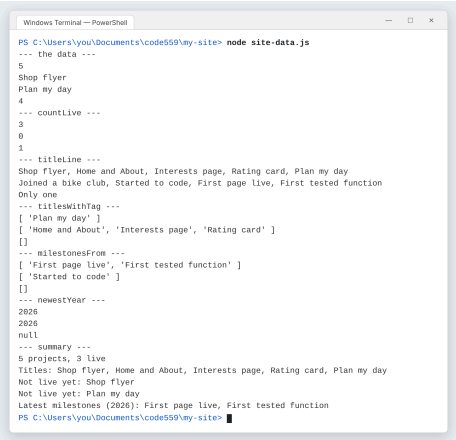
1. **`countLive`.** Write a function that takes a list of projects and returns how many of them are live.
2. **`titleLine`.** Write a function that takes a list of projects or milestones and returns all the titles as one line of text, with a comma and a space between them.
3. **`titlesWithTag`.** Write a function that takes a list of projects and a tag, and returns an array of the titles of the projects that have that tag.
4. **`milestonesFrom`.** Write a function that takes a list of milestones and a year, and returns an array of the titles of the milestones from that year.
5. **`newestYear`.** Write a function that takes a list of projects or milestones and returns the newest year in it, or `null` when the list is empty.

What it prints. Run from the `my-site` folder, `node site-data.js` printed exactly these 31 lines for the sample student, Sam:

```
--- the data ---
5
Shop flyer
Plan my day
4
--- countLive ---
3
0
1
--- titleLine ---
Shop flyer, Home and About, Interests page, Rating card, Plan my day
Joined a bike club, Started to code, First page live, First tested function
Only one
--- titlesWithTag ---
[ 'Plan my day' ]
[ 'Home and About', 'Interests page', 'Rating card' ]
[]
--- milestonesFrom ---
[ 'First page live', 'First tested function' ]
[ 'Started to code' ]
[]
--- newestYear ---
2026
2026
null
--- summary ---
5 projects, 3 live
Titles: Shop flyer, Home and About, Interests page, Rating card, Plan my day
Not live yet: Shop flyer
```


© 2026 559 Solutions LLC

```
Not live yet: Plan my day
Latest milestones (2026): First page live, First tested function
```



The finished file for the sample student, in one run. Each line matches its `// expect` comment. Arrays print with square brackets and single quotes, and an empty array prints as `[]`. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Your file prints the same shape with your own data:

- **One group of test lines for the data, then one group per function**, in the order of the descriptions. Each group starts with a heading line, such as `--- countLive ---`.
- **The data group** checks the arrays: the number of projects, the first title, the last title (found with `length - 1`), and the number of milestones.
- **At least three tests per function, and one of them an edge.** For `countLive` and `newestYear`, an empty list, `[]`. For `titleLine`, a list with one item. For `titlesWithTag`, a tag in capital letters. For `milestonesFrom`, a year with no milestones.
- **A summary group at the end** that uses the functions together: how many projects and how many are live, every title, one `Not live yet:` line for each project that is not live, and the titles of the milestones from the newest year.

What is inside the file.

- The first two lines are comments: the file name with one sentence about what it does, and how to run it.
- Four parts, each under its own heading comment, in this order: `// ===== The data =====`, `// ===== The functions =====`, `// ===== The tests =====` and `// ===== The summary =====`.
- Above each function, a comment with its name and its description, in your words.
- Above each group of tests, a comment that starts with `// expect` and lists your predictions, in order.
- Every function hands its answer back with `return`. No function has a `console.log()` inside it.
- `const` for every name that keeps its value, and `let` only for a value that changes as a loop runs: a counter, the loop's `i`, or the newest year so far. Never `var`. Comparisons use `===`, never `==`.

© 2026 559 Solutions LLC

The steps

Type every line. Do not paste. You may copy a test line and change only its argument, once you have typed the first test line of a group yourself.

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

2. **Plan on paper.** Start from the array of objects you wrote at the end of the lesson. On one page, list your projects. For each one, write its title, its year, one short sentence, its tags and whether it is live. On a second page, list your milestones with a title, a year and a sentence each. Read both lists once more: everything on them will be public, so leave out anything private, such as your full name, your school or where you live. Then, as in Module 7, make a small table for the five functions with three columns: **Name**, **Inputs** (one parameter name for each input) and **Output and its type**. For example, `countLive` takes `list` and returns a number. Do not write any code yet.

3. **Start the file.** In the VS Code Explorer, create `site-data.js` at the top level of `my-site`, next to `index.html`. Type these lines, then save with Ctrl+S:

   ```
   // site-data.js: my projects and milestones, with functions that summarize them
   // Run it from the my-site folder with: node site-data.js

   // ===== The data =====

   // ===== The functions =====

   // ===== The tests =====
   ```

   The data comes first, so every function and every test can use it. **Predict** before you run: what will `node site-data.js` print? Then type `node site-data.js` and press Enter. The prompt came straight back: only comments, so nothing to run.

4. **The data, and its first tests.** Under `// ===== The data =====`, leave one blank line and type the `projects` array. Put your own first project in place of Sam's:

   ```
   const projects = [
     {
       title: "Shop flyer",
       year: 2025,
       description: "A flyer page for a friend's bike shop.",
       tags: ["html"],
       live: false
     },
   ```


© 2026 559 Solutions LLC

```
  // Continue alone: one object for each of your other projects,
  // with the same five properties in the same order.
];
```

Each object sits inside its own curly braces, one property per line. Commas go **between** properties and **between** objects. Sam's file has no comma after the last property of an object, and none after the last object. The year and `live` have no quotes: one is a number and the other a Boolean, and your functions compare them that way. Add your other projects, then delete the two "Continue alone" lines.

Below the `projects` array, leave one blank line and type a `milestones` array the same way, with three properties per object: `title`, `year` and `description`. Sam's first milestone was:

```
const milestones = [
  {
    title: "Joined a bike club",
    year: 2024,
    description: "Weekly rides, and my first long route."
  },
  // Continue alone: your other milestones, oldest first.
];
```

Now test the data before you build anything on it. Under `// ===== The tests =====`, leave one blank line and type:

```
console.log("--- the data ---");
// expect 5, Shop flyer, Plan my day, 4
console.log(projects.length);
console.log(projects[0].title);
console.log(projects[projects.length - 1].title);
console.log(milestones.length);
```

Change the `// expect` comment to your own predictions: how many projects you typed, your first title, your last title, and how many milestones. `projects[0].title` reads from left to right: the array, then item 0, then its `title` (Core concept 9). `projects.length - 1` is the index of the last item, because counting starts at 0 (Core concept 2). Save and run. The sample printed `--- the data ---`, `5`, `Shop flyer`, `Plan my day` and `4`. If Node prints a `SyntaxError` and nothing else, a comma or a bracket is missing in the data. See Problem 5 in Part 4, *Troubleshooting*.

5. **`countLive`: the tests first, then the body.** Under the data tests, leave one blank line and type:

```
console.log("--- countLive ---");
// expect 3, 0, 1
console.log(countLive(projects));
```


© 2026 559 Solutions LLC

```
console.log(countLive([]));
console.log(countLive([{ live: false }, { live: true }]));
```

Change the first prediction to the number of your live projects. `[]` is an array with no items in it: its length is 0. That is the edge: a list with nothing to count. The last test passes a small array of two objects, made right inside the parentheses. Each object has only a `live` property, because that is the only one `countLive` reads. **Think this through:** the function does not exist yet. What will Node do? Save and run. The sample printed the whole data group, the heading line `--- countLive ---`, and then this (the `at` lines below it are left out):

```
C:\Users\you\Documents\code559\my-site\site-data.js:80
console.log(countLive(projects));
        ^

ReferenceError: countLive is not defined
```

You saw this message in Module 7. Your line number will differ, because your data has a different number of lines.

Now the body. Under `// ===== The functions =====`, leave one blank line and type the function with its comment:

```
// countLive: takes a list of projects and returns how many of them are live.
function countLive(list) {
  let count = 0;
  for (const item of list) {
    if (item.live) {
      count++;
    }
  }
  return count;
}
```

This is the counter pattern from Core concept 5: a counter before the loop, a check inside it, and the answer after it. `count` uses `let`, because it changes on every live project. The `for...of` loop puts each object in `item`, one lap at a time (Core concept 6). `item.live` is already `true` or `false`, so it can be the whole condition. The `return` comes **after** the loop's closing brace, so it runs once, when every item has been checked. Save and run. The sample printed `3`, `0` and `1` under the heading.

© 2026 559 Solutions LLC

6. **`titleLine` : push, then join.** Under the `countLive` tests, leave one blank line and type the tests first:

```
console.log("--- titleLine ---");
// expect the five project titles, the four milestone titles, Only one
console.log(titleLine(projects));
console.log(titleLine(milestones));
console.log(titleLine([{ title: "Only one" }]));
```

Write your own `// expect` comment. The second test passes the milestones: they have a `title` too, so the same function works for both arrays. The last test is the edge: a list with one item. Run once and see the `ReferenceError` for the new name. Then, under the `countLive` function, leave one blank line and start the function:

```
// titleLine: takes a list of projects or milestones and returns all the
// titles as one line of text, with a comma and a space between them.
function titleLine(list) {
  const titles = [];
  for (const item of list) {
    // Continue alone: push this item's title onto titles.
  }
  // Continue alone: return titles joined with a comma and a space.
}
```

`titles` starts as an empty array, and the loop fills it with `.push()`. It can be a `const`, because the name always points at the same array. Only the array's contents change (Core concept 3). After the loop, `.join(", ")` turns the array into one line of text (Core concept 7). Write the two missing lines, delete the "Continue alone" comments, save and run. The sample printed its five project titles on one line, its four milestone titles on the next, and `Only one`. With one item, there is nothing to put a comma between, so `.join()` adds none.

7. **`titlesWithTag` : a check inside the loop.** Under the `titleLine` tests, leave one blank line and type the tests first:

```
console.log("--- titlesWithTag ---");
// expect [ 'Plan my day' ], the three css titles, []
console.log(titlesWithTag(projects, "js"));
console.log(titlesWithTag(projects, "css"));
console.log(titlesWithTag(projects, "CSS"));
```

Use tags from your own data, and write your predictions. The capital letters in `"CSS"` are the edge: `.includes()` is exact, so `"CSS"` and `"css"` are different. This function returns an array, not text. In Node, `console.log()` shows an array with square brackets, a space inside each

© 2026 559 Solutions LLC

bracket, and single quotes around each string, such as `[ 'Plan my day' ]`. An empty array shows as `[]`. Run once and see the `ReferenceError`. Then, under `titleLine`, start the function:

```
// titlesWithTag: takes a list of projects and a tag, and returns an array
// of the titles of the projects that have that tag.
function titlesWithTag(list, tag) {
  const titles = [];
  for (const item of list) {
    // Continue alone: if item.tags includes tag, push item.title onto titles.
  }
  return titles;
}
```

`item.tags` is an array inside an object, so `item.tags.includes(tag)` reads from left to right: this project, its tags, and then the question "is `tag` in here?" The answer is `true` or `false`, which is exactly what an `if` needs. Write the `if` block, delete the comment, save and run. The sample printed `[ 'Plan my day' ]`, `[ 'Home and About', 'Interests page', 'Rating card' ]` and `[]`.

8. **`milestonesFrom`: create it alone.** This one has no code to copy, only the description from the brief. Do all five steps from Module 7 yourself. Steps 1 to 3 (name, inputs, output) are already in your table from step 2. Then:
   - In the tests part, under the `titlesWithTag` tests, type a heading line `--- milestonesFrom ---`, an `// expect` comment and three test lines: a year with two of your milestones, a year with one, and a year with none. Fill in the `// expect` comment before you run anything.
   - Run once and see the `ReferenceError`, as before.
   - In the functions part, under `titlesWithTag`, type the comment and the function. It has two parameters, the list and the year. Its shape is the same as `titlesWithTag`: an empty array, a loop, a check, a push, and a `return` after the loop. Only the check is different.

   Save and run. The sample tested 2026, 2025 and 2020, and printed `[ 'First page live', 'First tested function' ]`, `[ 'Started to code' ]` and `[]`. If every line prints `[]`, look at the check: it compares a number with a number only when the year in the test line and the year in the data both have no quotes.

9. **`newestYear`: the counting loop, and `null`.** Under the `milestonesFrom` tests, leave one blank line and type the tests first:

```
console.log("--- newestYear ---");
// expect 2026, 2026, null
console.log(newestYear(projects));
```


© 2026 559 Solutions LLC

```
console.log(newestYear([{ year: 2024 }, { year: 2026 }, { year: 2025 }]));
console.log(newestYear([]));
```

The second test hides the newest year in the middle of the list, so a function that only looks at the first or the last item fails it. The last test is the edge: an empty list has no newest year at all. Run once and see the `ReferenceError`. Then, under `milestonesFrom`, type the whole function:

```
// newestYear: takes a list of projects or milestones and returns the
// newest year in it, or null when the list is empty.
function newestYear(list) {
  if (list.length === 0) {
    return null;
  }
  let newest = list[0].year;
  for (let i = 1; i < list.length; i++) {
    if (list[i].year > newest) {
      newest = list[i].year;
    }
  }
  return newest;
}
```

Read it in three parts. First, an empty list returns `null` at once, the "no answer" on purpose from Module 7. Second, `newest` starts as the year of item 0, the best answer so far. Third, the loop checks every other item. It uses the counting `for` loop (Core concept 4) because it needs the positions: it starts at index 1, since item 0 is already in `newest`. Each time an item's year is greater, `newest` takes that year. That is why it is a `let`. Save and run. The sample printed `2026`, `2026` and `null`.

10. **The summary: use the functions together.** Under the last test line, leave one blank line and type:

```
// ===== The summary =====

console.log("--- summary ---");
console.log(projects.length + " projects, " + countLive(projects) + " live");
console.log("Titles: " + titleLine(projects));
for (const project of projects) {
  if (!project.live) {
    console.log("Not live yet: " + project.title);
  }
}
```


© 2026 559 Solutions LLC

```
const latest = newestYear(milestones);
const latestTitles = milestonesFrom(milestones, latest);
console.log("Latest milestones (" + latest + "): " + latestTitles.join(", "));
```

Read the `if` to yourself, out loud: "not live." `!` flips `false` to `true`, so the line prints for each project that is not live (Core concept 11). The last three lines hand the answer of one function to another: `newestYear` finds the year, and `milestonesFrom` uses it. **Think this through:** how many lines will the summary group print for your data? Write down your answer, then save and run. The sample printed five: `5 projects, 3 live`, the titles, two `Not live yet:` lines, and `Latest milestones (2026): First page live, First tested function`.

11. **Modify.** Make two changes, one at a time. Predict each before you run it, then undo it.

    - **Only live projects.** In `titlesWithTag`, change the condition to `item.tags.includes(tag) && item.live`. Save. Which test lines will change? Write them down, then run. In the sample, only the first `titlesWithTag` line changed, from `[ 'Plan my day' ]` to `[]`: `&&` needs both sides to be `true` (Core concept 11), and Plan my day is not live. The three css projects are all live, so that line stayed the same. Change the condition back, save and run.

    - **One more project.** Under the `milestones` array, leave one blank line and add a project with `.push()`:

      ```
      projects.push({
        title: "Site data",
        year: 2026,
        description: "My projects and milestones as data.",
        tags: ["js"],
        live: false
      });
      ```

      Think this through: which groups will change, and which will not? Write down your answer, then save and run. In the sample, the data group showed `6` and `Site data` as the last title. `countLive(projects)` stayed `3`, because the new project is not live. The first `titleLine` line, the first `titlesWithTag` line and the summary all grew. The summary printed `6 projects, 3 live` and a third `Not live yet:` line. Your `// expect` comments are now out of date, which is what a failing test looks like. Delete the `projects.push(...)` lines, save and run, and check that every line matches again. In Part 3 you add a real item and update the predictions too.

12. **Break it on purpose.** Two mistakes from the lesson. Make each one, run, look, and undo it before the next.

    - **One lap too many.** In `newestYear`, change `i < list.length` to `i <= list.length`. In the Exercise, this mistake printed one extra line with `undefined` and no error. **Think this through:** what will happen here, where each item is an object? Write it down, then save

© 2026 559 Solutions LLC

and run.

The sample printed the groups above `newestYear` as before, then the heading `--- newestYear ---`, and then this (the `at` lines below it are left out):

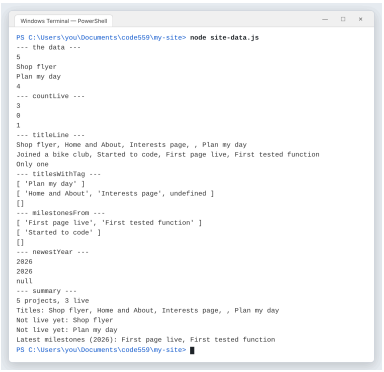
```
C:\Users\you\Documents\code559\my-site\site-data.js:122
    if (list[i].year > newest) {
                ^

TypeError: Cannot read properties of undefined (reading 'year')
```

The same one-character mistake as in the Exercise, but this time it is an error, and Node stops at the first `newestYear` test. On the last lap `i` equals the length, so `list[i]` is `undefined`. In the Exercise, `interests[4]` was `undefined` too, and printing `undefined` is allowed. Here the code reads a property **of** that `undefined`, and that is not allowed: there is no object to read from. The message names the property it tried to read, `year`. Change `<=` back to `<`, save and run.

- **Parentheses on a property.** In `newestYear`, change `list.length === 0` to `list.length() === 0`. Predict, save and run. The sample printed the groups above `newestYear` and then stopped with `TypeError: list.length is not a function`. `.length` is a property, a number you read. With parentheses, JavaScript tries to run that number, as `tags.length()` did in Core concept 8. Delete the parentheses, save and run.

13. **Debug: `undefined` with no error.** Here is a typing mistake that is easy to make in the data. In the object for your fourth project, change `title` to `titel`. Save. Before you run, write down which lines you expect to change. Then run.



One misspelled property name, no error message, and three wrong lines. Rating card is missing from both lines of titles, and the css array shows `undefined` where its title was. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Asking an object for a property it does not have gives `undefined`, quietly (Core concept 9). Every function that reads `item.title` got `undefined` for that one object.

© 2026 559 Solutions LLC

In the array, `undefined` has no quotes: it is the value, not the text "undefined". In the two lines of titles it does not show at all, because `.join()` turns it into empty text (MDN, *Array.prototype.join()*: "If an element is `undefined` or `null`, it is converted to an empty string"). A gap between two commas is the clue.

To find the object, count: the gap is where the fourth title should be, so the object is at index 3. Add this line at the end of the data tests, save and run:

```
console.log(projects[3]);
```

Node printed the whole object, and the misspelled name is right there:

```
{
  titel: 'Rating card',
  year: 2026,
  description: 'A card that asks visitors to rate a lesson.',
  tags: [ 'html', 'css' ],
  live: true
}
```

Change `titel` back to `title`, delete the `console.log(projects[3]);` line, save and run. Check every line against your comments. In your notebook, write one sentence on why `countLive` still gave the right answer while this bug was in the data.

14. **Explain it.** With every test passing, read the whole file to yourself, out loud, from top to bottom. For each function, say its parameters, what type of value it returns, and which test checks its edge. For each loop, say what is in the loop's variable on the first lap and on the last lap. For `newestYear`, say why the loop starts at 1 and why the check uses `<`, not `<=`. If there is a line you cannot explain, reread the Core concept it comes from.

15. **Looking ahead to Module 9.** No page uses `site-data.js` yet. In Module 9 you learn to build page items from an array of objects: a `for...of` loop visits each object, and for each one, `document.createElement()` makes a new list item and `append()` puts it on the page. It is the same shape as the loops you wrote here, with a page element where your `.push()` was. The habits carry over too: check the data first, read each property with a dot, and test with an empty list.

16. **Check the file.** Tick each box.

    - [ ] `site-data.js` is in `my-site`, next to `index.html`, not in `practice-08`
    - [ ] The prompt ends in `my-site>`, and `node site-data.js` prints seven groups with no error
    - [ ] Every test line matches its `// expect` comment, in order

© 2026 559 Solutions LLC

- [ ] Each function has at least three tests, one of them an edge: an empty list, one item, a tag in capitals, or a year with no milestones
- [ ] Every project has the same five properties, every milestone the same three, and no `undefined` appears in the output
- [ ] Every function hands back its answer with `return`; there is no `console.log()` inside any function
- [ ] `newestYear` uses `<`, not `<=`, and `.length` has no parentheses
- [ ] The extra `projects.push(...)` line from step 11 and the `console.log(projects[3]);` line from step 13 are gone
- [ ] Nothing in the data is private: titles, years and short sentences you are happy for anyone to read

17. **Save it in Git.** Go to Part 3, *Finishing work in Git*, and save the file with the save routine.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Every test in `site-data.js` passes. Now you save it to GitHub with the same save routine as in Modules 1 to 7: `git status`, `git add .`, `git commit`, `git push`, then check the live site. Two things are new this time. The file holds real facts about you, so you read it as a stranger would before you publish it. And after the first commit you add one more milestone, which changes what your tests should print, and you save that change as a second commit.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 to 7.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **Run the tests one last time.** Type `node site-data.js` and read every line against its `// expect` comment. Look for the signs of the mistakes from Project steps 11 to 13: an `undefined`, a gap between two commas, an extra project you did not plan, or a `TypeError`. Check that the `projects.push(...)` lines and the `console.log(projects[3]);` line are gone. Commit only a file whose tests you have just run and seen pass.

3. **Read your data as a stranger would (NEW: real data is public).** Your earlier scripts printed made-up plans and page names. This file holds real facts about you. Once you push it, anyone who has the address can read it, the same as your pages. Read every `title` and every `description` once more, slowly. Keep what you would put on your About page. Take out anything you would not tell a stranger, such as your full name, your school, your street, or a year that gives away your age. If you change the data, run `node site-data.js` again and update any `// expect` comment that no longer matches.

4. **See what changed.** Type:

   ```
   git status
   ```

   Under **Untracked files** you should see one line: `site-data.js`. Git has never saved it before. Your `practice-08` folder never shows up here, because it is outside `my-site`.

5. **Stage the change.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. (A warning that mentions LF and CRLF is harmless.) Run `git status` again. Under **Changes to be committed** you should now see `new file: site-data.js`.

© 2026 559 Solutions LLC

6. **Commit.** Type:

```
git commit -m "Add site-data.js with my projects, milestones and five tested
functions"
```

   On the sample site, Git printed `1 file changed, 180 insertions(+)` on the second line and `create mode 100644 site-data.js` on the third. The 180 is the number of lines in the sample file, the data, comments, tests and blank lines included. Your commit id and line count will differ.

7. **Push.** Type:

```
git push
```

   The last line ends in `main -> main`.

8. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 7 commit (or your latest one) under it.

9. **Add a milestone, and save again (NEW: when the data changes, the tests change).** Finishing this project is a milestone too. In the `milestones` array, add one more object at the end. Put a comma after the closing `}` of the object before it. The sample added:

```
  {
    title: "First data file",
    year: 2026,
    description: "My projects and milestones, tested in Node."
  }
```

   Think this through: which lines of the output will change? Write down your answer before you run anything. In the sample, four lines changed: the number of milestones in the data group (`4` became `5`), the line of milestone titles, the 2026 line in the `milestonesFrom` group, and the last line of the summary.

   Your code did not change, but the right answers did. So update every `// expect` comment that no longer matches. The sample changed three:

```
// expect 5, Shop flyer, Plan my day, 5
// expect the five project titles, the five milestone titles, Only one
// expect the three 2026 titles, [ 'Started to code' ], []
```


© 2026 559 Solutions LLC

Save, run `node site-data.js`, and check every line against its comment. Then type `git status`. This time it lists `modified: site-data.js` under **Changes not staged for commit**. Run the routine again, with a message that names the change:

```
git add .
git commit -m "Add a milestone: First data file"
git push
```

On the sample, the commit printed `1 file changed, 8 insertions(+), 3 deletions(-)`. The three changed `// expect` comments count as three lines deleted and three added, and the new milestone added the other five.

`git log --oneline` now shows both commits on top, the newest first. From now on, each time you finish a project or reach a milestone, you can add it here the same way: change the data, update the predictions, run the tests, then commit.

10. **Check the live site.** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/` and press F5. Your pages look exactly as before, because no page loads `site-data.js`. Then open `https://your-username.github.io/my-site/site-data.js`. As with your other scripts, the browser shows your code as text or offers to download it, and nothing runs. This is what a stranger sees, so read it once more there.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your two new commits on top. `node site-data.js` prints every group with no error, and every line matches its `// expect` comment. Your live pages look the same as before, and the live address of `site-data.js` shows your code without running it.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose in the sample `site-data.js` and run with Node 22. The messages were captured on Linux, with the folder path changed to the course folder; on Windows, Node prints the same words with your own path. The `at` lines under each message are left out here, and your line numbers will differ. After each fix, save, run `node site-data.js` again from the `my-site` folder, and check every line against its `// expect` comment. Problem 4 prints no error at all. Your tests are what catch it.

1. TypeError: Cannot read properties of undefined

Symptom. Some groups print, then the file stops. With `projects[projects.length].title` in place of `projects[projects.length - 1].title` in the data tests, the sample printed `--- the data ---`, `5` and `Shop flyer`, and then:

```
C:\Users\you\Documents\code559\my-site\site-data.js:135
console.log(projects[projects.length].title);
                                      ^

TypeError: Cannot read properties of undefined (reading 'title')
```

With `tag:` in place of `tags:` in one project, the sample stopped at the first `titlesWithTag` test with `TypeError: Cannot read properties of undefined (reading 'includes')`, pointing at the line `if (item.tags.includes(tag)) {`. Project step 12 shows the same message for `year`.

Why. The code asked for a property **of** something that is `undefined`. There are two usual ways to get there. An index past the end gives `undefined` (Core concept 2): with 5 projects, `projects[5]` does not exist. Or a property the object does not have gives `undefined` (Core concept 9): a project with `tag` has no `tags`. Reading `undefined` is allowed. Reading a property of it is not, so Node stops. The name in brackets at the end of the message is the property it tried to read, and the caret points at that name in your line.

Fix. Look at what comes just before the property named in the message. If it holds an index, check the index: the last item is at `length - 1`, and a loop checks `i < list.length`, not `<=`. If it is a property such as `item.tags`, check the spelling of that property in every object of your data, capital letters included. `console.log()` the object to see its real property names, as in Project step 13.

© 2026 559 Solutions LLC

2. TypeError: x is not a function

Symptom. Some groups print, then the file stops. With `.join(", ")` added after `titleLine(projects)` in the summary, the sample printed every group down to `5 projects, 3 live`, and then:

```
C:\Users\you\Documents\code559\my-site\site-data.js:172
console.log("Titles: " + titleLine(projects).join(", "));
                                             ^

TypeError: titleLine(...).join is not a function
```

With `list.length()` in `newestYear`, the message was `TypeError: list.length is not a function` (Project step 12). And with `let titles = "";` in place of `const titles = [];` in `titleLine`, it was `TypeError: titles.push is not a function`.

Why. Parentheses mean "run this." Each of these put `()` after something that cannot run. `.length` is a property: a number, not a method (Core concept 8). `titleLine` returns text, already joined, and text has no `.join()` method: `.join()` belongs to arrays. And `""` is empty text, not an empty array, so it has no `.push()`. The `(...)` in the first message stands for the call, `titleLine(projects)`, whose answer was a string.

Fix. Find the name just before `is not a function`, and ask what kind of value it holds. A property, such as `.length`, loses its parentheses. A method has to belong to the right type: `.push()` and `.join()` need an array, and `[]` makes an empty one. If a function already returns text, use the text as it is.

3. TypeError: Assignment to constant variable.

Symptom. Some groups print, then the file stops. With `const count = 0;` in `countLive`, the sample printed the data group and the `--- countLive ---` heading, and then:

```
C:\Users\you\Documents\code559\my-site\site-data.js:74
      count++;
           ^

TypeError: Assignment to constant variable.
```

With `const i = 1` in the `for` line of `newestYear`, the message was the same, and the caret pointed at the `i++` part of the line.

Why. A `const` cannot get a new value (Module 6). `count++` and `i++` both give the name a new value, 1 more than before. Notice when the error comes: not at the `const` line, but the first time the code tries to change the value. In `countLive`, that is at the first live project. In the loop, it is right after the first

© 2026 559 Solutions LLC

lap, when the step `i++` runs.

Fix. Use `let` for a counter, for the `i` of a counting `for` loop, and for a value such as `newest` that changes as the loop runs. Keep `const` for everything else, including the arrays you `.push()` onto and the `item` of a `for...of` loop: each lap makes a new `item`, so it never changes.

4. A number or a list that is too small, with no error

Symptom. No error, but some answers are smaller than your predictions. Two mistakes gave this in the sample.

- With `return count;` moved up inside the loop, just under the `if` block, the `countLive` group printed `0`, `undefined` and `0` instead of `3`, `0` and `1`. The summary said `5 projects, 0 live`.
- With the year of one milestone typed as `"2026"`, in quotes, the 2026 line of the `milestonesFrom` group printed only `[ 'First page live' ]`, and the last summary line lost a title too.

Why. `return` ends the function at once, even in the middle of a loop (Module 7). With `return` inside the loop, the function stops at the end of the first lap and hands back the count so far. The sample's first project is not live, so the answer was `0`. With an empty list there is no lap at all, the function reaches its end with no `return`, and the answer is `undefined`. The year in quotes is text, and `===` treats different types as different (Module 6): `"2026" === 2026` is `false`, so that milestone is never picked.

Fix. Put `return` after the loop's closing brace, so it runs once, when every item has been checked. Check the indentation: the `return` lines up with the `for`, not with the `if`. For the year, take the quotes off: every `year` in your data is a number. Both bugs pass some tests and fail others, which is why each function has more than one test.

5. SyntaxError messages

Symptom. Nothing prints at all, not even the first heading line. Node shows a line of your file, a caret or carets, and a message that starts with `SyntaxError`.

Why. Node could not read the file as JavaScript, so it ran none of it. In this file, most of these come from the data: many brackets, braces and commas in a row. Each message below was made on purpose in the sample file.

© 2026 559 Solutions LLC

Message	Usual cause	Where to look
`Unexpected token '{'`	A comma missing between two objects (or a `)` missing before `{`; see Module 7, Problem 6)	The `}` just above the named line
`Unexpected identifier 'year'`	A comma missing at the end of the property above	The line just above the named line
`Invalid shorthand property initializer`	`=` in place of `:` in a property, such as `year = 2025`	The named line
`Unexpected token 'const'`	The `]` missing at the end of the `projects` array	The end of the array just above
`Unexpected token ']'`	The `}` missing at the end of the last object	The line just above the named `]`
`Invalid or unexpected token`	A missing quote, so a string never ends	The string under the carets

Fix. Fix the one thing the row points to, save, and run. If the message changes, read the new one from the top: one missing comma can hide another mistake below it. For messages about the braces of a function or an `if`, Problem 6 in the Module 7 packet has a longer table: the rules are the same.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 9. If a line is a 1 or a 2, reread that Core concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can make an array, read an item by its index counting from 0, and find the last item with length - 1	☐	☐	☐	☐
I can add an item with .push(), and explain why it works on an array made with const	☐	☐	☐	☐
I can write a counting for loop, name its start, check and step, and spot an off-by-one error from <=	☐	☐	☐	☐
I can put an if inside a loop to count or collect items, with the return placed after the loop	☐	☐	☐	☐
I can choose between for...of and the counting for loop, and say why	☐	☐	☐	☐
I can use .includes() and .join(), and predict what capital letters and an empty array do to them	☐	☐	☐	☐
I can tell a property from a method, and fix "is not a function" when parentheses are on a property	☐	☐	☐	☐
I can write an array of objects, read its properties with a dot, and find a misspelled property from the undefined it gives	☐	☐	☐	☐
I can combine checks with &&, || and !, and predict the result before I run it	☐	☐	☐	☐

Three questions to answer in writing

1. In the Exercise, changing `<` to `<=` printed one extra line, `4: undefined`, with no error. In Project step 12, the same change in `newestYear` stopped the file with a `TypeError`. Explain the difference.

2. Pick one test from your project, or one prediction from the Exercise, where your first answer was wrong. What did you write down, what did Node or the Console show, and what do you understand now that you did not before?

© 2026 559 Solutions LLC

3. A friend wrote this from the description *"Write a function that takes a list of projects and returns the titles of the live ones as one line of text."* With Sam's `projects`, what does `console.log(liveLine(projects));` print? What two changes fix it? And if you fix only the line with `return`, what does it print then?

```
function liveLine(list) {
  const titles = [];
  for (const item of list) {
    if (item.live) {
      titles.push(item.titel);
    }
    return titles.join(", ");
  }
}
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **`tags.length()` and `tags.push("js")`.** `.length` is a property: a value stored on the array, the number of items. You read it, so it takes no parentheses. Parentheses mean "run this," and a number cannot run, so `tags.length()` stops with `TypeError: tags.length is not a function`. `.push()` is a method: a function that belongs to the array. It needs parentheses to run, and the item to add goes inside them. Without the parentheses, `tags.push` is only a reference to the function, as with `showMessage` in Module 7, and nothing is added. (Core concept 8)
2. **The last of six projects.** It is at index 5, because the first one is at index 0. Write it as `projects[projects.length - 1]`. With six projects, `projects.length - 1` is 5. Push a seventh, and the same code reads index 6 with no change. (Core concept 2)
3. **What value does `n` hold?** `2`. The loop checks each place with `p.visited || p.free`, which is `true` when at least one side is `true`. City library: visited, so `true`, and `n` becomes 1. Art museum: not visited and not free, so `false || false`, which is `false`. River park: not visited, but free, so `true`, and `n` becomes 2. (Core concepts 5 and 11)

Common misconception

"An array with 4 items ends at index 4." It ends at index 3. The index counts steps from the start, so the first item is 0 steps away and the fourth is 3 steps away. The length, 4, is how many items there are, not a position: the last index is always `length - 1`. In the Concept 2 picture, `pages[4]` printed `undefined`, because there is nothing there. The same belief causes the off-by-one error: a loop with `i <= list.length` makes one lap too many. In the Exercise that lap printed `4: undefined`. In Project step 12 it read `.year` from that `undefined` and stopped with a `TypeError`. (Core concepts 1, 2 and 4; Exercise step 7; Project step 12)

Exercise checks

These are the results with the example data, and with `"cooking"` pushed in step 2, as in the lesson's picture. With your own interests, the shape is the same.

- **Step 1.** `3` and `music`. `interests.length` is 3, and `interests[interests.length - 1]` is `interests[2]`, the last item. (Core concepts 1 and 2)
- **Step 2.** The `.push()` line prints nothing by itself. It adds the item at the end, and the length is now 4. (Core concept 3)

© 2026 559 Solutions LLC

- **Step 3.** Four lines: `0: drawing`, `1: cycling`, `2: music` and `3: cooking`. (Core concept 4)
- **Step 4.** `true` for an item you have and `false` for one you do not, then `drawing | cycling | music | cooking`. (Core concept 7)
- **Steps 5 and 6.** Ten lines in all: `3`, `music`, the four loop lines, `true`, `false`, the joined line, and `Next free trip: River park`. City library is free but already visited, so `!place.visited` is `false`. Art museum is not visited, but it is not free. Only River park makes both sides of `&&` true. (Core concepts 9 and 11)
- **Step 7.** The loop prints a fifth line, `4: undefined`, and the run has 11 lines. There is no error. On the last lap `i` is 4, the same as the length, and `interests[4]` does not exist. (Core concepts 2 and 4)
- **Step 8.** One possible answer:

  ```
  let freeCount = 0;
  for (const place of places) {
    if (place.free) {
      freeCount++;
    }
  }
  console.log(freeCount);
  ```

  It prints `2`, as the last of 11 lines: City library and River park are free. `freeCount` is a `let`, because it changes on each free place. (Core concept 5)
- **Step 9.** With `||`, all three places print: `Next free trip: City library`, `Next free trip: Art museum` and `Next free trip: River park`. With `||`, one true side is enough. City library is free. Art museum and River park are not visited, so `!place.visited` is already `true` for them. With the step 8 loop below it, the run has 13 lines. (Core concept 11)

Self-assessment questions

1. **`undefined` in the Exercise, a `TypeError` in the project.** In both, the extra lap reads one index past the end, and that gives `undefined`. In the Exercise, the code only printed that value: `interests[4]` joined to `"4: "` gave the line `4: undefined`. Printing `undefined` is allowed, so there was no error. In `newestYear`, each item is an object, and the code reads a property of it: `list[i].year`. On the extra lap, `list[i]` is `undefined`, and `undefined` has no properties at all. Node cannot read `.year` from nothing, so it stops with `TypeError: Cannot read properties of undefined (reading 'year')`. The cause is the same `<=`; the result depends on what the code does with the missing item. (Core concepts 2, 4 and 9; Project step 12)
2. **A prediction you got wrong.** There is no single answer. A good answer quotes what you wrote down before the run, quotes what Node or the Console really showed, and names the idea you understand now. For example: that `.push()` hands back the new length, not the array, or that

© 2026 559 Solutions LLC

`.join()` shows an `undefined` item as nothing at all.

3. **The friend's `liveLine`.** It prints an empty line, with no error. There are two mistakes. First, `return` is inside the loop, so the function ends at the end of the first lap. Sam's first project, Shop flyer, is not live, so nothing was pushed, and `[].join(", ")` is empty text. Second, `item.titel` is misspelled, so every push adds `undefined`. The two changes: move `return titles.join(", ");` below the loop's closing brace, and change `titel` to `title`. With only the `return` moved, it prints `, , ` (two commas and two spaces): three `undefined` items, which `.join()` turns into empty text. With both changes, it prints `Home and About, Interests page, Rating card`. (Core concepts 7 and 9; Project step 13; Problem 4 in Part 4)

Project checks

- **Step 2.** The sample's plan for the functions:

Name	Inputs	Output and its type
`countLive`	`list`	a number, like `3`
`titleLine`	`list`	a string, like `Shop flyer, Home and About`
`titlesWithTag`	`list`, `tag`	an array of strings, like `[ 'Plan my day' ]`
`milestonesFrom`	`list`, `year`	an array of strings, or `[]` when there are none
`newestYear`	`list`	a number, like `2026`, or `null`

- **Step 3.** Nothing prints, and there is no error.
- **Step 4, continue alone.** The sample's two arrays are at the top of the whole file, at the end of this section. With them, the data group printed `5`, `Shop flyer`, `Plan my day` and `4`.
- **Step 5.** Before the function: the data group, the heading, then `ReferenceError: countLive is not defined`, pointing at the first `countLive` test (line 80 in the sample). After it: `3`, `0` and `1`. The empty list gives `0`, not an error: the loop makes no laps, and `count` is still 0 when the function returns it.
- **Step 6, continue alone.** The sample's `titleLine`:

```
// titleLine: takes a list of projects or milestones and returns all the
// titles as one line of text, with a comma and a space between them.
function titleLine(list) {
  const titles = [];
  for (const item of list) {
    titles.push(item.title);
```


© 2026 559 Solutions LLC

```
  }
  return titles.join(", ");
}
```

- **Step 7, continue alone.** The sample's `titlesWithTag`:

```
// titlesWithTag: takes a list of projects and a tag, and returns an array
// of the titles of the projects that have that tag.
function titlesWithTag(list, tag) {
  const titles = [];
  for (const item of list) {
    if (item.tags.includes(tag)) {
      titles.push(item.title);
    }
  }
  return titles;
}
```

- **Step 8, create alone.** The sample's tests and function. The tests go in the tests part, the function in the functions part.

```
console.log("--- milestonesFrom ---");
// expect the two 2026 titles, [ 'Started to code' ], []
console.log(milestonesFrom(milestones, 2026));
console.log(milestonesFrom(milestones, 2025));
console.log(milestonesFrom(milestones, 2020));
```

```
// milestonesFrom: takes a list of milestones and a year, and returns an
// array of the titles of the milestones from that year.
function milestonesFrom(list, year) {
  const titles = [];
  for (const item of list) {
    if (item.year === year) {
      titles.push(item.title);
    }
  }
  return titles;
}
```

- **Step 9.** `2026`, `2026` and `null`. For `[{ year: 2024 }, { year: 2026 }, { year: 2025 }]`, `newest` starts at 2024, becomes 2026 on the first lap, and stays 2026 on the second, because 2025 is not greater.

© 2026 559 Solutions LLC

- **Step 10.** Five lines: `5 projects, 3 live`, `Titles: Shop flyer, Home and About, Interests page, Rating card, Plan my day`, `Not live yet: Shop flyer`, `Not live yet: Plan my day` and `Latest milestones (2026): First page live, First tested function`.

- **Step 11.** With `&& item.live`, only the first `titlesWithTag` line changes, from `[ 'Plan my day' ]` to `[]`.

  With the extra project pushed, six lines change and one is added. In the data group, `6` and `Site data`. The first `titleLine` line ends in `, Site data`, and the first `titlesWithTag` line is `[ 'Plan my day', 'Site data' ]`.

  The summary starts `6 projects, 3 live`, its titles line ends in `, Site data`, and a third line `Not live yet: Site data` is added. The run has 32 lines.

- **Step 12.** With `<=`: `TypeError: Cannot read properties of undefined (reading 'year')` at the first `newestYear` test. With `list.length()`: `TypeError: list.length is not a function`, at the same test. In both runs, every group above `newestYear` prints as before.

- **Step 13.** Three lines change: the first `titleLine` line and the summary's titles line each show a gap, `Interests page, , Plan my day`, and the css line of `titlesWithTag` shows `undefined` in place of `'Rating card'`. `countLive` still gives `3`, because it reads only `item.live`, and that property was spelled right.

- **Step 16.** If you followed the brief, every box is ticked. The most common misses: a `// expect` comment that still holds the sample's predictions instead of yours, and a leftover line from step 11 or step 13. Here is the whole sample file, 180 lines:

```
// site-data.js: my projects and milestones, with functions that summarize them
// Run it from the my-site folder with: node site-data.js

// ===== The data =====

const projects = [
  {
    title: "Shop flyer",
    year: 2025,
    description: "A flyer page for a friend's bike shop.",
    tags: ["html"],
    live: false
  },
  {
    title: "Home and About",
    year: 2026,
    description: "My first two pages, with one nav bar.",
    tags: ["html", "css"],
    live: true
```


© 2026 559 Solutions LLC

```
  },
  {
    title: "Interests page",
    year: 2026,
    description: "A card grid that fits any screen.",
    tags: ["html", "css"],
    live: true
  },
  {
    title: "Rating card",
    year: 2026,
    description: "A card that asks visitors to rate a lesson.",
    tags: ["html", "css"],
    live: true
  },
  {
    title: "Plan my day",
    year: 2026,
    description: "A script that plans any hour of the day.",
    tags: ["js"],
    live: false
  }
];

const milestones = [
  {
    title: "Joined a bike club",
    year: 2024,
    description: "Weekly rides, and my first long route."
  },
  {
    title: "Started to code",
    year: 2025,
    description: "One hour a day, most days."
  },
  {
    title: "First page live",
    year: 2026,
    description: "My Home page went live on GitHub Pages."
  },
  {
    title: "First tested function",
    year: 2026,
    description: "Tests first, then the body."
  }
];
```


© 2026 559 Solutions LLC

```
// ===== The functions =====

// countLive: takes a list of projects and returns how many of them are live.
function countLive(list) {
  let count = 0;
  for (const item of list) {
    if (item.live) {
      count++;
    }
  }
  return count;
}

// titleLine: takes a list of projects or milestones and returns all the
// titles as one line of text, with a comma and a space between them.
function titleLine(list) {
  const titles = [];
  for (const item of list) {
    titles.push(item.title);
  }
  return titles.join(", ");
}

// titlesWithTag: takes a list of projects and a tag, and returns an array
// of the titles of the projects that have that tag.
function titlesWithTag(list, tag) {
  const titles = [];
  for (const item of list) {
    if (item.tags.includes(tag)) {
      titles.push(item.title);
    }
  }
  return titles;
}

// milestonesFrom: takes a list of milestones and a year, and returns an
// array of the titles of the milestones from that year.
function milestonesFrom(list, year) {
  const titles = [];
  for (const item of list) {
    if (item.year === year) {
      titles.push(item.title);
    }
  }
  return titles;
}
```


© 2026 559 Solutions LLC

```
// newestYear: takes a list of projects or milestones and returns the
// newest year in it, or null when the list is empty.
function newestYear(list) {
  if (list.length === 0) {
    return null;
  }
  let newest = list[0].year;
  for (let i = 1; i < list.length; i++) {
    if (list[i].year > newest) {
      newest = list[i].year;
    }
  }
  return newest;
}

// ===== The tests =====

console.log("--- the data ---");
// expect 5, Shop flyer, Plan my day, 4
console.log(projects.length);
console.log(projects[0].title);
console.log(projects[projects.length - 1].title);
console.log(milestones.length);

console.log("--- countLive ---");
// expect 3, 0, 1
console.log(countLive(projects));
console.log(countLive([]));
console.log(countLive([{ live: false }, { live: true }]));

console.log("--- titleLine ---");
// expect the five project titles, the four milestone titles, Only one
console.log(titleLine(projects));
console.log(titleLine(milestones));
console.log(titleLine([{ title: "Only one" }]));

console.log("--- titlesWithTag ---");
// expect [ 'Plan my day' ], the three css titles, []
console.log(titlesWithTag(projects, "js"));
console.log(titlesWithTag(projects, "css"));
console.log(titlesWithTag(projects, "CSS"));

console.log("--- milestonesFrom ---");
// expect the two 2026 titles, [ 'Started to code' ], []
console.log(milestonesFrom(milestones, 2026));
console.log(milestonesFrom(milestones, 2025));
console.log(milestonesFrom(milestones, 2020));
```


© 2026 559 Solutions LLC

```
console.log("--- newestYear ---");
// expect 2026, 2026, null
console.log(newestYear(projects));
console.log(newestYear([{ year: 2024 }, { year: 2026 }, { year: 2025 }]));
console.log(newestYear([]));

// ===== The summary =====

console.log("--- summary ---");
console.log(projects.length + " projects, " + countLive(projects) + " live");
console.log("Titles: " + titleLine(projects));
for (const project of projects) {
  if (!project.live) {
    console.log("Not live yet: " + project.title);
  }
}
const latest = newestYear(milestones);
const latestTitles = milestonesFrom(milestones, latest);
console.log("Latest milestones (" + latest + "): " + latestTitles.join(", "));
```

This is the file before Part 3. In Part 3 you add one more milestone and update three `// expect` comments.

NEXT

Module 9: DOM and Events.
Free lesson: 559solutions.com/code559/module-09-dom-events/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC